

Best Practices in Data Management for Cryo-EM Labs

JASON C. PORTA, PH.D.

08 JUNE 2024

PORTA031@UMN.EDU

INTRODUCTION:

Data storage issues:

Macromolecular structure determination by cryo-electron microscopy (Cryo-EM) requires the collection of many images of biological specimens. Furthermore, the introduction of direct electron detectors (DED), the development of new microscope hardware and the availability of automated data collection software triggered a significant increase in the resolvability of Cryo-EM structures and the rate by which they are determined. This is made possible mainly due to the DED's high detective quantum efficiency which requires the collection of images as a series of frames at a fast rate, making them capable of counting incident electrons. Images that are collected in this manner are saved as "movies", which are single micrographs that are calculated by summing up each frame per image. Despite giving the ability to correct for beam-induced motion of the sample specimen, movie files require significantly more storage space on disk, and this has created a serious issue for many labs as the size of Cryo-EM datasets have significantly grown and are being collected at much faster rates. The major bottleneck here is the finite amount of disk space that is allocated to each lab. Here, the worst-case scenario is when all data collection must be halted due to a lack of disk space, and although it may seem a simple matter of requesting more disk space from IT, the reality is that with better management of our filesystem, this should not be necessary. By implementing good practices in data management, the cost of big data in Cryo-EM can be significantly reduced.

Datasets produced by both single-particle analysis (SPA) and cryo-electron tomography (cryo-ET) often consist of thousands of images. A single-particle dataset for example will often contain 5,000 to 10,000+ raw, non-drift corrected movie files. When one considers that a single raw TIF file is approximately 0.45 GB, a dataset with 10,000 frames will occupy 4.5 TB of disk space. Keep in mind too that this is the minimum required disk space as each movie file must be averaged and drift corrected, as well as processed further for structure determination. This process will produce many files starting from drift-corrected images to refined reconstruction(s). For example, while parsing through the /panfs/home/shafen area of the computing cluster, there was a data collection directory within the 'shcryosparc' folder with 11,724 raw TIF frames. This single data set (amongst many) is therefore occupying $(11,724 \times 0.45 \text{ GB}) \approx 4.3 \text{ TB}$ of disk space. Furthermore, many jobs within the data processing pipeline also produce many big files (i.e., Refine3D), most of which are from intermediary iterations that are no longer needed once the job run has finished. For example, when Legion and Appion are used for SPA data collection the Appion implementation of MotionCorr2 will perform drift correction on the averaged micrographs (*en.mrc) for both a dose-weighted (*en-a-DW.mrc) and a non-dose-weighted (*en-a.MRC) file, each of which are approximately 55 0MB. And unlike other implementations of

MotionCorr2 (i.e. RELION), the user is not given the option to only write the dose-weighted *or* non-dose-weighted micrographs. This means that for each micrograph submitted for drift correction there will be two micrographs written to disk, one that is dose-weighted and another that is not. Therefore, one should first avoid producing unnecessary and redundant files when possible, and/or to delete any files that are not needed for further processing (i.e., *en-a.mrc or *en-a-DW.mrc).

Leginon Micrographs from a Single Microscope Session			
File Type	No. Micrographs	Total Size	Disk Space Occupied
*.mrc (all micrographs)	1,369,563	75T	14%
*gr/sq/hl/fa.mrc	162,963	0.5T	0.1%
*en.mrc	416,365	23T	4.2%
*en-a.mrc	366,065	20T	3.7%
*en-a-DW.mrc	289,316	16T	3.0%

Table 1. Files accumulated from a single data collection session. Highlighted in red are all micrographs collected during a single-particle experiment in Leginon. In green are the files that are not needed post data collection, which should be immediately removed from disk and in purple are the files necessary for data processing.

An example is given above in **Table 1** that shows the files that should eventually be deleted along with how much disk space they currently occupy (highlighted in green). Also, don't forget that a single microscope session generates many lower magnification images (i.e., grid, hole, square and FFT images) that are no longer needed once data collection is complete. In general, it is always best to set up your jobs to where you are minimizing the amount of unnecessary file output. From the above table one can clearly see how a significant amount of a group's total disk storage allotment can become cluttered by unneeded files and why it is critical for each user to be mindful about what they are storing on the filesystem. Here are some more reasons that disk storage issues with big data occur:

- Poorly curated data that is stored in multiple places (or in the wrong places) on the filesystem (i.e. EMPIAR, '/panfs/home/shafen' (cluster), and cold storage¹¹).
- Lack of maintaining and organizing project folders resulting in a significant number of large files that are sup. This typically takes the form of aborted job runs, intermediate files and unused job results from processing attempts using numerous data processing software.
- Each user has their own system for naming and organizing their data which oftentimes is very time consuming and stressful to find all the relevant files when a new lab member is taking over a project. Thus, a standardized system of naming and storing our data can make a big difference.
- Not following or sticking with the best procedures for data management.

¹ The cold storage system is operated locally at the Hormel Institute and will become available soon. I am currently working with Jeff from IT on getting this setup for the lab.

- Copying large files and/or entire datasets from one area of the filesystem to another thereby creating *another* copy of their dataset instead of simply creating symbolic links.

Good practices in data management can significantly reduce the cost of big data in Cryo-EM and will be covered in more detail in this document. These practices include the minimization and elimination of data beyond its' stage of usefulness, keeping clear and concise notebooks and records up to date and the implementation and adherence to group-wide standards for organization and archival of data. However, for this guide to be effective it will require each person to take the time to initially clean-up their project directories and to be regimented about implementing these practices in the future.

Example From the Lab Filesystem:

As of this writing, the Hafenstein lab is currently allotted approximately 775T of disk space located at /panfs/home/shafen, and our quota is unlikely to keep increasing in the future. Currently, 90% (694T) of the allotted disk quota is occupied. Also, there are times when users experience job failures and aborts due to the disk being full, causing a complete halting of both data collection and processing for everyone in the group. Please keep in mind though success in this context is *entirely* dependent on every lab member following these instructions and doing so regularly. This cannot be stressed enough.

Procedural Outline for Managing Your Files:

1. Find the location of and create an initial inventory of your files and folders. There are three main areas that you should focus on. They are:
 - a. /panfs/home/shafen —mostly project folders
 - b. /panfs/home/shafen/shared —shared files between lab members
 - c. Local disk on PCs, etc. —data stored on your personal PCs

It is important to go through each folder you own recursively such that you are parsing all directories and not just the files and directories at the top level.

2. Find and delete all files and folders that are unnecessary and unneeded and that do not require long-term storage post-publication. This includes cryoSPARC projects, which can be managed and deleted in the browser interface. For RELION jobs, you will focus mostly on the individual job folders (i.e., Class2D, Refine3D) and the trash folders. More details on this are given in Chapter 3.
3. Transfer your data that is not actively being processed to the lz published data that is no longer being used to cold storage or EMPIAR.
4. Organize your active data and associated project folders using the guidelines presented in this document.

Before You Begin:

The purpose of having a computing cluster is intended to provide the computational resources (compute nodes, parallelization of computationally intensive jobs etc) necessary to process cryo-EM data in a timely efficient manner, so ideally the only data that should be stored in the LSI storage area are files needed to run jobs for active projects. The cluster (with the locker as an exception) was not meant to be used as a storage area for inactive and non-job-related folders. The challenge here lies in adopting a mindset where you are always maintaining proper file system practices while taking measures to reduce the amount of data from accumulating. This is where keeping an updated inventory of your files and folders will help tremendously. However, before organizing your active projects, be sure to do your best to remove all unneeded files and folders from the file system. This includes smaller sized files as well since despite their small size, they are numerous and only act to create more clutter, and by effect a more difficult to navigate file system. Another thing to keep in mind too is that although the practices, particularly section 2.5, which covers how your data should be organized are not a rigid set of rules meant to be followed *exactly* as outlined here. Given the unique nature of each project, some flexibility is expected. Finally, on the topic of file deletion be aware that should you unintentionally delete anything by accident, you have 30 days to have that data recovered by LSI IT. After 30 days, the data will have cycled off the daily backup drives and are no longer recoverable. So always be mindful of what you are deleting, and should there be any doubt, always consult with your P.I. first.

ORGANIZING YOUR DATA:

Locating Your Project Files and Directories on The Cluster:

- In the lab's directory identify and print a list of your files and folders to the terminal by issuing the command below. See chapter 5 more a more detailed list of commands to aid in this:

```
$ ls -l | grep user
```

Tip: for large output to the terminal, it can often be helpful to redirect your output to a file using the redirect operator '>'. Also, I often find printing the output to have on paper at my side is helpful as well.

- Print a list of all your files and folders and datasets including a description of each folder's contents so that when a new lab member picks up your project in the future. For a more detailed list of commands to aid in this, please see chapter 5.
- Make a spreadsheet of all your project folders and datasets with the relevant information required for data processing. Your inventory spreadsheet should contain:

☐ Project Name:	Caveolin
☐ Sample name:	Cav1-Myc 8S Complex
☐ Location of data on the filesystem:	/lsi/groups/mohilab/Jason/Cav1_Myc
☐ Date of data collection:	16 September 2020
☐ Microscope:	Glacios (BSB)
☐ Detector pixel size (Å):	0.98
☐ Total electron dose (e ⁻ /Å ²):	56.5
☐ Number of micrographs collected:	4,207
☐ Status:	Manuscript submitted
☐ Size of project folder:	6.2T

- For your datasets that are either published or not expected to be processed soon, transfer them to the data locker. Only store raw data to the locker as it too fills quickly with unnecessary data processing output that can always be regenerated if need be.
- Organize your data by following the directory tree structure outlined at the end of this chapter.

Finding and deleting unnecessary files and directories:

- Delete any projects and/or workspaces from failed and/or unneeded processing workflows written by all other software packages you processed your data with.
- Delete any unneeded files, folders and failed jobs in each of your project folders. It is important to be aware that many software packages write hidden files and directories often, so be sure to include the '-a' option flag with the 'ls' command as some programs accumulate a significant number of files in hidden directories (i.e., '.phenix' directories).
- Delete any tutorial data and its output. These can always be reobtained from an online source if needed in the future.
- Transfer your raw image frames to the data locker upon completion of drift-correction: These are the largest of the image files and are no longer needed for further processing after motion correction is complete.
- Delete any non-dose-weighted images (or the dose-weighted ones if you didn't use those): Some drift-correction programs (i.e., MotionCor2 in Appion) by default write out the non-dose-weighted images as well as the dose-weighted ones, literally doubling the number of micrographs in your project directory. Since you only want to use the dose-weighted images, delete the others.
- Delete project folders from software packages that were not used for further processing: For example, there are a significant number of cisTEM project folders in our disk area, many of

which have not been accessed in a long time, some even years. So, if cisTEM (or whatever other program) didn't produce good results, there is no need to keep it.

- Find and delete duplicate files: There is now a script (/lsi/groups/mohilab/scripts/find_duplicate_files.sh) that will find duplicate files recursively by matching their size. Since many files (i.e., particles.star) share the same name, yet contain unique content, filename matching can be dangerous. Also, I decided not to include a file deletion component as this is something that should be done carefully by the user.
- Delete unneeded log files and other temporary files, such as logs, compute job output, etc.

```
$ #!/usr/bin/bash
$
$ awk -F'/' '{
$   f = $NF
$   a[f] = f in a? a[f] RS $0 : $0
$   b[f]++ }
$   END{for(x in b)
$       if(b[x]>1)
$           printf "Duplicate Filename: %s\n%s\n",x,a[x] }' <(find . -
type f) \
$               > `find_duplicate_files_`whoami`.out'
$ echo `|-----FIND ROUTINE COMPLETED ON: ``date``-----|`
```

Figure 1. Bash script for finding duplicate files in the current directory. To search a specific directory, replace the '.' on line 9 following the find command with the directory name. This script can be found in /lsi/groups/mohilab/scripts/find_duplicate_files.sh.

Management of Leginon datasets (/panfs/home/shafen):

This directory contains the output images written by Leginon. This includes all frame averaged micrographs taken at all magnifications during screening (i.e., grid, square, hole, focus and exposure mode magnifications) as well as drift-corrected images from Appion ('leginon' folder), and raw image frames ('frames' directory). This area of the filesystem is *not* for processing data. Any micrographs and/or frames should be symbolically linked to your project folder in /lsi/groups/mohilab.

Cleaning up 'leginon' directory:

- ☐ Delete any non-dose-weighted micrographs.
- ☐ Delete all project folders of bad grid screens and failed data collection sessions.
- ☐ Delete all low-magnification images from the square, hole and focus modes. Before doing this, I would recommend downloading an image of the grid atlas for your notes from the Leginon web interface.
- ☐ For folders of published data, you can delete your drift-corrected images as long as the raw data frames are stored on the locker.

- ☐ If there are any project folders in this area of the file system, move them to the /lsi/groups/mohilab area. There should not be any project folders in the 'cryoem_automation' directory.

Cleaning up the 'frames' directory:

- ☐ Delete all project folders of bad grid screens and failed data collection sessions.
- ☐ For folders containing published frames, transfer them to the data locker (/nfs/locker/lsi-mohilab). Don't forget to include the gain reference files in the 'references' directory in case the frames need to be re-processed later.
- ☐ If there are any project folders in this area of the file system, move them to the /lsi/groups/mohilab area. There should not be any data processing folders in the 'cryoem_automation' directory.

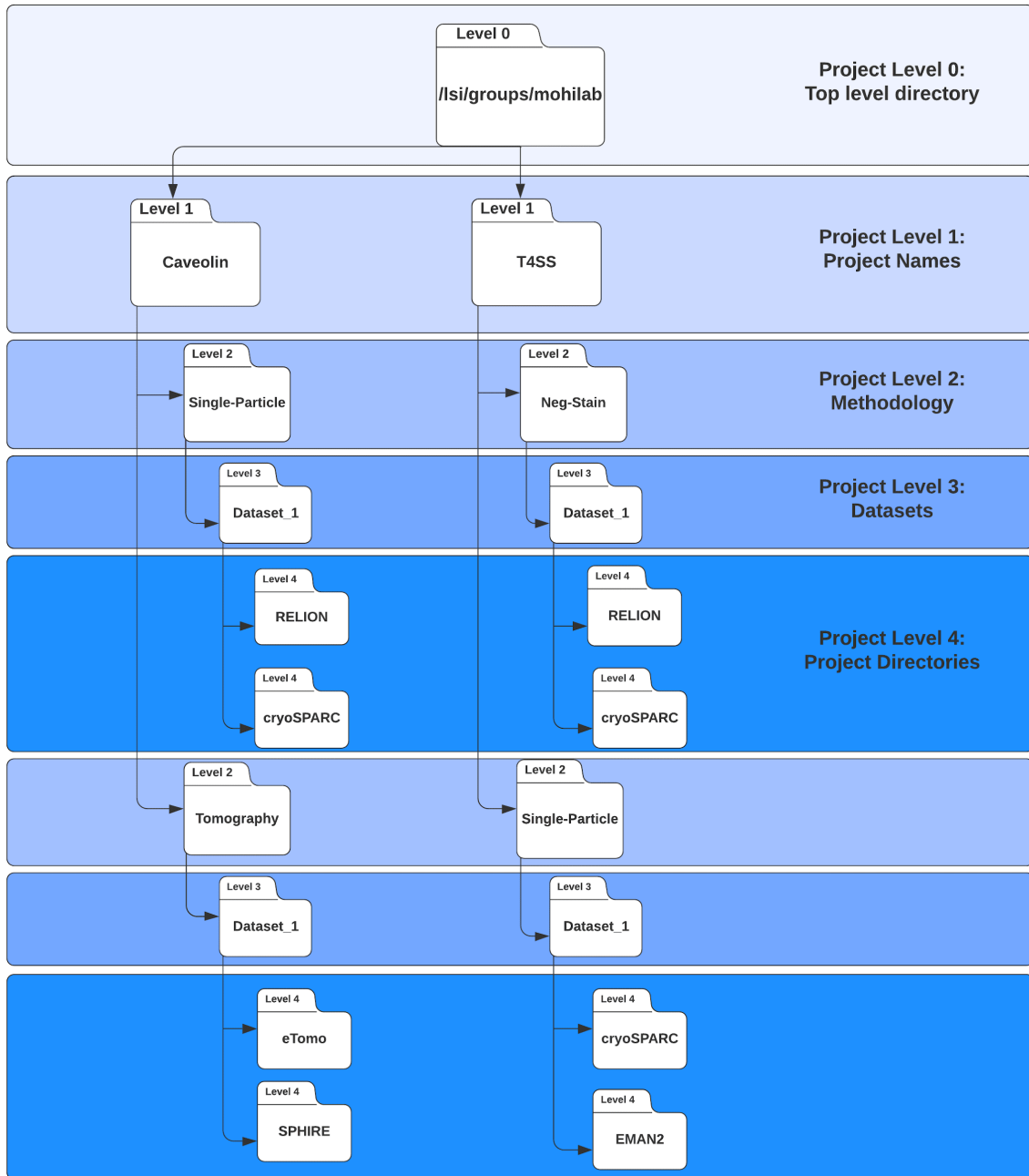


Figure 2. Template for storing data being actively processed on the LSI cluster.

Managing CryoSPARC Projects

Managing your projects in cryoSPARC is not as tedious as in the previous sections due to having a user-friendly web interface. When creating new projects and workspaces, create them in your /lsi/groups/mohilab/user folder. Like above however, symbolically link your frames (using the 'ln -s' command) from the 'cryoem_automation/frames' area. You can monitor your disk usage for each project in the 'Data Management' section of the 'Resource Manager' on the web interface.

Items:

- ☐ Delete any projects and/or workspaces from failed and/or unneeded processing workflows.
- ☐ For projects of published data, save the relevant files such as final refined coordinates (cryoSPARC's 3D-Flex software should be available soon), refined maps (sharpened and unsharpened), FSC half maps and masks etc. At the most, keep only the jobs that were used in the data processing pipeline to produce the published data.
- ☐ Within each workspace, delete all failed and aborted jobs. Be careful what you delete, and that the job's output isn't required as input for a later job in the published data processing pipeline.
- ☐ In the projects section of the web interface, click on each of your projects and select 'Clear Intermediate Results'.
- ☐ Delete all rejected micrographs and their associated frames from all your 'Manual Curation' jobs. See section 5 on how to do this in a more automated fashion.

The code below will create a list of rejected micrographs from the cryoSPARC manual curation routine and then delete them from the filesystem. I would recommend first commenting out the

last six lines and running the program to generate a list of the files to be deleted. I would then carefully examine the file to be certain the correct files are listed for deletion. Finally, uncomment the last six lines and run the script again. To use this script, log into *mohi-gpu01.lsi.umich.edu* via SSH and set up the cryoSPARC Python environment by typing these two commands at the command line. When ready, then invoke the script by typing:

```
$ eval $(/usr/local/cryosparc2/cryosparc2_master/bin/cryosparcm
env)
$ export PYTHONPATH=${CRYOSPARC_ROOT_DIR}
$ python delete_rejected_cryosparc_micrographs.py
```

```
###-----###
###Delete micrographs that were rejected during manual curation###
###
###Usage: python delete_rejected_micrographs.py
###
###When prompted give the full path and filename from cryoSPARC###
###-----###

#Import dataset module for manipulating .cs files
from cryosparc_compute import dataset

#Initialize the dataset object
exposure_dset = dataset.Dataset()

#Request filename and path from user
dataset_path = input("Please enter the filename including the full path:
")

#Load the .cs file
exposure_dset.from_file(dataset_path)

#Print out a sample of all values in this field
exposure_dset.data['micrograph_blob/path']

#The following will write all the filenames to a text file that
#can be piped to a unix delete command
with open("exposures_to_delete.txt", 'w') as openfile:
    for file_to_delete in exposure_dset.data['micrograph_blob/path']:
        openfile.write(file_to_delete + '\n')

#Delete the files sequentially from the newly written text file
import os
for file_to_delete in exposure_dset.data['micrograph_blob/path']:
    try:
        os.remove(file_to_delete)
    except:
        print("Unable to delete {}".format(file_to_delete))
    continue
```

Managing RELION Data

Managing your RELION projects is mostly done from the command line and organizing your project folders is a very useful means by which to navigate your folders. Several RELION jobs create a significant number of large files that in the case of multiple job runs for (which is more common than not), a lot of disk space can become occupied by a single RELION project.

Items:

- ☐ Delete any projects and/or workspaces from failed and/or unneeded processing workflows.
- ☐ Delete the contents of the 'Trash' folder. Note that deleting the job in the user interface does not actually delete the files. The 'Trash' folder must have its contents deleted from the command line with the 'rm' command.
- ☐ Symbolically link only your raw frames from the 'cryoem_automation/frames' directory.
- ☐ Delete unused job output, especially for programs that write out micrographs (i.e., MotionCor), and particle stacks (i.e., Extract). Often it is necessary to run multiple instances of these jobs until the best job parameters are determined. However, once that happens, delete the old jobs, leaving just the final "good" job output).
- ☐ When doing drift-correction with MotionCor2 in RELION, there is an option to also write the non-dose-weighted images. Do not select this (it is turned off by default) option. Search your old and current project folders and delete the non-dose-weighted images.

Command Line Resources:

This final section contains a list of various ways in I found the Linux 'find' command to be useful in my search of my own data management misdeeds:

```
#---Command #1---#
#Find all MRC files for user jporta, the -type f option flag
will find only files and exclude symbolically linked files:

$ find . -name "*en-a.mrc" -type f -user jporta

#---Command #2---#
#Find all symbolically linked MRC files owned by user jporta:

$ find . -name "*.mrc" -type l -user jporta

#---Command #3---#
#Find all broken symlinked files for user jporta:

$ find . -name "*.mrc" -xtype l -user jporta
```

```

#---Command #4---#
#Find all MRC files recursively between 50-250MB, which is the
size range of micrographs in #MRC format and ignores the smaller
MRC files that are not microscope images:

$ find . -name "*.mrc" -type f -size +$((50*1024*1024))c -size -
$((250*1024*1024))c

#---Command #5---#
#Same as command #4 but will also print the size of each file in
human readable format:

$ find . -type f -name "*.mrc" -size +$((50*1024*1024))c -size -
$((250*1024*1024))c -exec du -sh {} \;

#---Command #6---#
#Same as the previous command, but excludes the
'cryoem_automation' directory, which is the #one place where
micrographs should be stored:

$ find . -name "*.mrc" -type f -size +$((50*1024*1024))c -size -
$((250*1024*1024))c \
-not -path "./cryoem_automation/*"

#---Command #7---#
#Same as the previous command, except it searches for multiple
file types, in this example 'find' looks for the three types of
Leginon micrographs (*en.mrc, *en-a.mrc and *en-a-#DW.mrc),
RELION MotionCor2 (*frames.mrc) and cistEM Unblur (*_0.mrc):

$ find ./ -type f \( -name \*frames.mrc -o -name \*en.mrc -o -
name \*en-a.mrc -o -name \*en-a-DW.mrc -o -name \*_0.mrc \) -
size +$((50*1024*1024))c -size -$((250*1024*1024))c -not -path
"./cryoem_automation/*" > all_micrographs_for_group_mohilab.txt
$ find . -type f \( -name \*.mrc -o -name \*.tif -o -name
\*.mrns \) -size +$((50*1024*1024))c -size -$((250*1024*1024))c
-not -path "./cryoem_automation/*"

#---Command #8---#
#Find and print a list of the 25 largest directories:

$ du -xk | sort -n | tail -25

#---Command #9---#
#These three commands produce varying ways to print out a
directory tree diagram to the terminal. This is useful for

```

better understanding your directory hierarchy. I would highly recommend redirecting the output to a file with the > operator as the output can be large:

```
$ find . | sed -e 's;[^\/*];|____;g;s;____|; |;g'
$ find . -print | sed -e 's;[^\/*];|____;g;s;____|; |;g'
$ find . -type d -print | sed -e 's;[^\/*];|____;g;s;____|; |;g'
```

#---Command #10---#

#Print all directories in a tree pattern two levels deep from the current directory:

```
$ tree -d jporta -L 2 -prune
```

('-d' will print only directories, '-L n' Allows the user to define how deep into the filesystem to parse and the '--prune' flag will not print empty directories.